MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (Honours)/ MGU-BBA (Honours)/ MGU-BCA (Honours)

REGULAR EXAMINATION October 2026

Fifth SEMESTER

Discipline Specific Core Course (DSC) - MG5DSCECT300, MG5DSCECC300 - DIGITAL DESIGN USING VERILOG

(2024 ADMISSION ONWARDS)

PART A

1. [b] Decimal
2. [d] OR
3. [a] case
4. [c] assign
5. [c] assign Sum = A^B^Cin
6. [c]Gate
7. [b]Force the counter to a known initial state
8. [b]it stores a binary state and can be used as a basic memory element
9. [b]Bitwise NOT
10. [a] 30 ns

PART B

11. Verilog is a Hardware Description Language (HDL) used to design, model, simulate, and verify digital electronic circuits and systems. It is called a Hardware Description Language because:
It describes the structure and behaviour of digital circuits.
It represents hardware components such as logic gates, multiplexers, counters, and flip-flops.
It allows designers to simulate and test circuit operation before implementing the hardware.
It is used to design and implement digital circuits on devices such as FPGA and ASIC.
[K] / [CO1]

12. A 3-to-8 decoder is a combinational logic circuit that has 3 input lines and 8 output lines. For each input combination, only one output is HIGH (1), while all the other outputs remain LOW (0). A 3-to-8 decoder converts a 3-bit binary input into one of 8 active outputs using the corresponding Boolean expressions. Let the inputs be A, B, C and the outputs be Y_0 to Y_7.
The 3 inputs can represent $2^3 = 8$ different combinations.
Each input combination activates one unique output.
The decoder is used in memory address decoding and digital systems.
Verilog Code (Dataflow Modeling)
verilog

```
module decoder3to8 (
   input A, B, C,
```

```
    output [7:0] Y
);
    assign Y[0] = ~A & ~B & ~C;
    assign Y[1] = ~A & ~B &  C;
    assign Y[2] = ~A &  B & ~C;
    assign Y[3] = ~A &  B &  C;
    assign Y[4] =  A & ~B & ~C;
    assign Y[5] =  A & ~B &  C;
    assign Y[6] =  A &  B & ~C;
    assign Y[7] =  A &  B &  C;
endmodule
```

[U] / [CO2]

13. A Verilog testbench is used to simulate and verify the correct operation of a digital circuit before hardware implementation. The main components of a Verilog testbench are:
Module Declaration: Defines the testbench module. It usually has no input or output ports.
Instantiation of DUT: The Design Under Test (DUT) is the circuit being tested and is instantiated inside the testbench.
Stimulus Generation: Provides input signals to the DUT to test its functionality.
Monitoring and Verification: Observes the output signals and checks whether the outputs match the expected results. [K] / [CO3]

14. Concurrency means that multiple hardware operations execute simultaneously. In Verilog, independent always blocks and continuous assign statements operate concurrently. Verilog supports concurrency to model the parallel behaviour of digital hardware, whereas conventional programming languages generally follow sequential execution.
Difference between Concurrency and Sequential Execution:

Concurrency in Verilog	**Sequential Execution**
Multiple operations occur simultaneously.	Statements execute one after another.
Represents the parallel operation of hardware circuits.	Commonly used in conventional programming languages like C.
Independent always blocks execute concurrently.	Statements generally execute in the order written.
Example: Two registers can be updated at the same clock edge.	One statement executes after the previous statement completes.

[U] / [CO1]

15. *XOR Gate (Exclusive-OR):* The output of an XOR gate is HIGH (1) when the two inputs are different.
Boolean expression:

$$Y = A \oplus B = \bar{A}\,B + A\,\bar{B}$$

Verilog dataflow expression:
verilog

```
module XOR_GATE(input A, B, output Y);
    assign Y = A ^ B;
endmodule
```

XNOR Gate (Exclusive-NOR): The output of an XNOR gate is HIGH (1) when the two inputs are the same.
Boolean expression:

$$Y = A \odot B = AB + \bar{A}\ \ \bar{B}$$

Verilog dataflow expression:
verilog
```
module XNOR_GATE(input A, B, output Y);
   assign Y = ~(A ^ B);
endmodule
```
[U] / [CO2]

16. A 4-bit shift register is a sequential circuit consisting of four flip-flops connected in series. It stores 4 bits of data and shifts the data by one position for every clock pulse. At every positive edge of the clock, the input bit sin enters the register, and the existing bits shift one position towards the output. The most significant bit q[3] is available at the serial output sout.

Serial-In Serial-Out (SISO): Data is entered one bit at a time through the serial input. At every active clock edge, the data shifts by one position. The data is obtained one bit at a time from the serial output. A 4-bit SISO register requires four clock pulses to load four bits of data.

Verilog Code:
verilog
```
module SISO (
   input clk, sin,
   output sout
);
   reg [3:0] q;

   always @(posedge clk)
      q <= {q[2:0], sin};

   assign sout = q[3];
endmodule
```
[U] / [CO3]

PART C

17. A Full Subtractor is a combinational logic circuit that subtracts two bits B and B_{in} from input bit A. The full subtractor performs subtraction of three input bits and produces the difference and borrow outputs. The gate-level Verilog design uses logic primitives and wire elements, while the testbench verifies all eight input combinations. It has three inputs and two outputs.
Inputs: A(minuend), B(subtrahend), B_{in}(borrow input).
Outputs: D(difference), B_{out}(borrow output).
The Boolean expression for the difference is:

$$D = A \oplus B \oplus B_{in}$$

The Boolean expression for the borrow output is:

$$B_{out} = \bar{A}\,B + \bar{A}\,B_{in} + BB_{in}$$

Verilog Code Using Gate-Level Modelling - Gate-level modelling uses logic gate primitives such as xor, and, or, and not. The intermediate signals are declared using wire.
verilog
```
module full_subtractor(
   input A, B, Bin,
   output D, Bout
);
```

```
    wire x1, nA, w1, w2, w3;

    xor (x1, A, B);
    xor (D, x1, Bin);

    not (nA, A);
    and (w1, nA, B);
    and (w2, nA, Bin);
    and (w3, B, Bin);
    or  (Bout, w1, w2, w3);

endmodule
```

A testbench applies all eight possible input combinations and displays the outputs.

[A] / [CO2]

18. Verilog is a Hardware Description Language (HDL) used to design, simulate, and verify digital electronic circuits. The basic structure of a Verilog program consists of modules, ports, declarations, and statements. These components help describe, simulate, and verify digital circuits efficiently.

The general structure of a Verilog program is:

verilog

```
module module_name (ports);

    // Port declarations
    // Signal declarations
    // Continuous assignments
    // Procedural statements

endmodule
```

A Verilog program consists of one or more modules. Each module describes a particular digital circuit or hardware component.

Main Components

Modules

- A module is the basic building block of a Verilog program.
- It defines a digital circuit or hardware component.
- A module begins with module and ends with endmodule.

Example:

verilog

```
module AND_GATE (A, B, Y);
```

Ports

- Ports are used to communicate signals between a module and its external environment.
- The three main port types are:
    - input – receives signals.
    - output – sends signals.
    - inout – supports bidirectional signals.

Example:

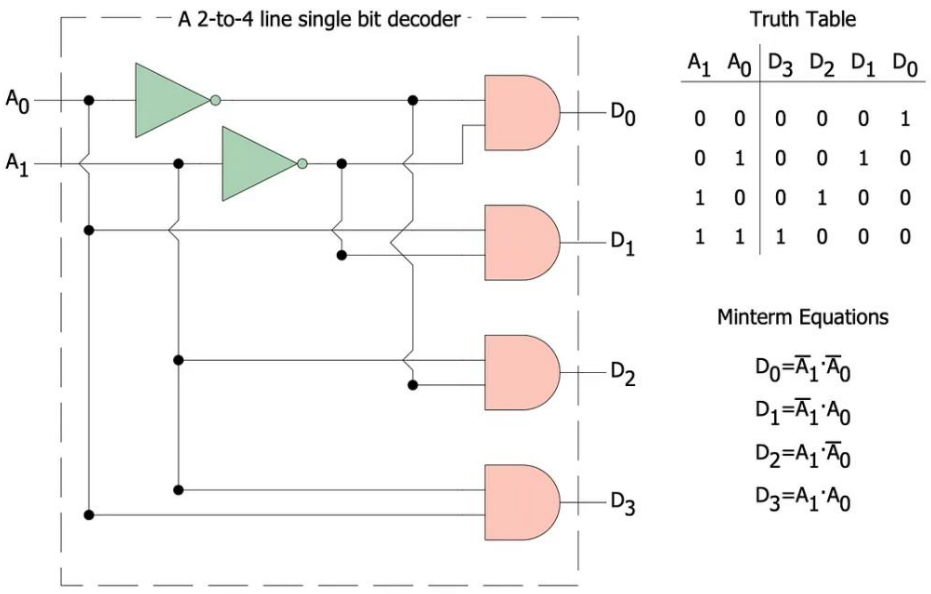
verilog

```
input A, B;
output Y;
```

Declarations
Declarations define the type and size of signals used in a module. Common declarations include:
wire – represents connections between hardware components.
reg – stores a procedural variable value and can represent combinational or sequential logic.
integer – represents an integer variable, often used in testbenches.
Example:
verilog
wire temp;
reg Q;
Statements
Statements describe the operation of the circuit.
assign – performs continuous assignment.
always – describes procedural behaviour.
initial – executes a procedural block once at the start of simulation. [U] / [CO1]

19. (a) Data types
data: Net (wire)
count: Register (reg)
i: Integer (integer)
voltage: Real (real)
delay: Time (time)
(b) Number of bits
data: 8 bits
count: 4 bits
(c) integer: Loop counters and arithmetic calculations
real: Storing floating-point/decimal numbers in simulation
time: Storing simulation time
(d) To differentiate physical wiring (wire) from storage elements (reg)
To separate hardware design constructs from simulation/testbench variables
To support both continuous assignments and procedural assignments
(e) Declarations - 16-bit signal: wire [15:0] signal;
8-bit procedural variable: reg [7:0] variable [U] / [CO1]

20. A decoder is a combinational circuit that converts binary information from n input lines into a maximum of 2^n unique output lines. A 2×4 decoder takes 2 input bits and activates one of four output lines depending on the input combination. It's widely used in memory address decoding, data routing, and control circuits.



Behavioral modeling describes the operation of a circuit using procedural statements such as always and case.

verilog

```
module decoder2to4(
   input [1:0] A,
   output reg [3:0] Y
);

always @(*) begin
   case (A)
      2'b00: Y = 4'b0001;
      2'b01: Y = 4'b0010;
      2'b10: Y = 4'b0100;
      2'b11: Y = 4'b1000;
      default: Y = 4'b0000;
   endcase
end

endmodule
```

Applications - Memory address decoding / Instruction decoding / Display systems / Data routing. [U] / [CO3]